Parcours avancé — Un serveur solide

Pour qui : les étudiants qui codent déjà un serveur · **Quand :** après les TD 9 à 12, faits vite · **Où :** dans ton projet des TD

Quatre exercices indépendants, dans l'ordre que tu veux.

A1 — Un seul type pour le site et le serveur

Le type d'un film est écrit une seule fois, dans un dossier `shared`. Le site et le serveur l'importent tous les deux.

C'est réussi quand : renommer un champ dans ce type fait échouer `npm run build`, du côté du site comme du côté du serveur, tant que les deux ne sont pas corrigés.

Si tu as fini : le type du corps d'un POST se déduit du type du film, sans l'identifiant ni « vu ».

A2 — Vérifier avec un schéma

Le corps du POST et celui du PATCH sont vérifiés par un schéma Zod. Le type du film se déduit du schéma.

C'est réussi quand : un film invalide reçoit le statut 400, avec un message pour chaque champ en erreur. Il ne reste aucun test de validation écrit à la main.

Si tu as fini : le formulaire utilise le même schéma, avant l'envoi.

A3 — Tester ton API

Avec le lanceur de tests de Node, sans autre outil, teste ton API : la liste, un film, 404, 400, un ajout accepté (201) et un ajout refusé (400). Le serveur de test démarre sur un port libre, et s'arrête à la fin.

C'est réussi quand : `node --test` passe. Si tu casses la vérification du titre, un test échoue.

Si tu as fini : chaque test part de données neuves : l'ordre des tests ne change rien.

A4 — Les erreurs au même endroit

Ajoute une route `/api/panne`, qui lève une erreur exprès. Un seul gestionnaire d'erreurs répond alors le statut 500, avec un message en JSON, et écrit l'erreur dans le terminal. Une adresse inconnue qui commence par `/api` répond 404, en JSON.

C'est réussi quand : après `/api/panne`, le serveur répond toujours aux autres routes. Aucune route n'a son propre bloc `try`.

Si tu as fini : un corps JSON mal formé reçoit le statut 400, avec un message en JSON, et pas 500.